Case study

SWE-smith and SWE-Gym

How to build coding RL environments at scale

Learning outcomes

After this case study, you can:

1. Name the parts of a coding task and its grader.
2. Explain how SWE-Gym collected real tasks and built environments.
3. Explain how SWE-smith makes tasks by breaking working code.
4. Compare the cost, scale and realism of the two methods.
5. Describe how the grader selects training data.
6. Plan a small coding RL environment of your own.

The problem: good coding tasks are scarce

Demand	Labs want coding RL environments more than any other kind.
Good news	Unit tests give a clear, automatic reward.
Bad news	Each task needs a working copy of the repository, with every dependency installed.
Old datasets	A few thousand tasks at most, from 11 or fewer repositories.

Anatomy of one coding task

Repository	A real codebase, fixed at one commit.
Issue text	A bug report. This is the prompt.
FAIL_TO_PASS	Tests that fail now. The fix must make them pass.
PASS_TO_PASS	Tests that pass now. The fix must not break them.
Environment	A Docker image with the code and all its dependencies.

Part 1

SWE-Gym

Real issues, environments built by hand

SWE-Gym: from 64,689 tasks to 2,438

358	Python repositories pass the filters.
64,689	raw tasks from their pull requests.
11	repositories get runnable environments.
2,438	tasks pass the unit-test check.

SWE-Gym: building the environments

Method	People set the dependencies for each task by hand, from the files of that date.
Check	The real fix must pass more tests than the old code.
Effort	About 200 human hours and 10,000 CPU core hours.
Storage	One Docker image per task: 6 TB in total.

SWE-Gym: training on passing rollouts

1. Strong models (GPT-4o, Claude 3.5 Sonnet) try the tasks in the OpenHands agent.
2. The tests grade each attempt. Only passing attempts are kept: 491 rollouts.
3. Qwen2.5-Coder models of 7B, 14B and 32B are fine-tuned on those rollouts.

The grader chooses the training data.

SWE-Gym: results on SWE-bench Verified

Model	Before	After	Gain
7B	1.8%	10.6%	+8.8 points
14B	4.0%	16.4%	+12.4 points
32B	7.0%	20.6%	+13.6 points

Self-training failed: the 32B model’s own rollouts cut its Lite score from 15.3% to 8.7%.

SWE-Gym: a verifier picks the best try

A 32B verifier learns from 2,636 rollouts, half passing and half failing. It scores each new attempt.

Attempts (k)	Any attempt passes	Verifier’s pick passes
1	20.6%	20.6%
8	37.8%	29.8%
16	42.8%	32.0%

The gap between the two columns is room for a better verifier.

SWE-Gym: what the authors learned

More rollouts	Scores kept rising. They did not level off at 491 rollouts.
The bottleneck	Compute to sample rollouts, not the number of tasks.
Easy-task bias	Easy tasks give the most passing rollouts. A cap of 2 per task helped.
Limits	Python only, few repositories, and environments built by hand.

Part 2

SWE-smith

Make the bugs yourself, at 20 times the scale

SWE-smith: the core idea

> Start from working code. Break it on purpose. Keep the break only if a test catches it.

Old way: one environment per task, built from the code of that date.

New way: one environment per repository, shared by hundreds of tasks.

SWE-smith: one environment per repository

Pick	Popular PyPI packages with 1,000+ stars, minus SWE-bench's 12. Result: 128 repositories.
Install	An agent installs the repository and runs its tests, in at most 100 steps.
Check	A person checks the steps: about 8 minutes per repository. At least 80% of tests must pass.
Package	One Docker image per repository. All its tasks share it.

SWE-smith: five ways to make a bug

Strategy	Kept	Tasks	Cost each
LM modifies code	56.0%	17,887	0.38¢
Random code change	40.2%	15,641	0.00¢
Combine two bugs	96.9%	10,092	0.00¢
LM rewrites function	35.0%	4,173	3.93¢
Undo a real PR	33.8%	2,344	5.53¢

“Kept” is the share of candidate bugs that broke a passing test.

SWE-smith: check the bug, then write the issue

Keep rule	The bug must break at least one test that passed before.
Time limit	A test run that takes more than 2 minutes is dropped.
Issue input	An LM sees the bug’s diff, one failing test and the test output.
Issue output	A GitHub-style bug report with code to reproduce it. About 2.5¢ each.

SWE-smith: the scale and the cost

50,137
tasks from 128 repositories

$1,360
total cost to build the tasks

295 GB
storage, against 6 TB for SWE-Gym

One author built it in about 20 hours. SWE-Gym needed about 200 human hours.

SWE-smith: from tasks to training data

1. Claude 3.7 Sonnet tries 8,686 tasks in the SWE-agent harness. Limits: 75 steps, $2 per try.
2. Of 17,906 tries, 36% pass the tests.
3. Keep at most 3 passing rollouts per task: 5,016 rollouts.
4. Fine-tune Qwen2.5-Coder 7B and 32B on them.

SWE-smith: results for SWE-agent-LM

40.2%

32B on SWE-bench Verified, one try

30.7%

32B on SWE-bench Lite

15.2%

7B on SWE-bench Verified

For context: SWE-Gym’s 32B model scored 20.6%, and R2E-Gym’s 32B model 34.4%.

SWE-smith: what made the data good

Bug type	Undone real PRs helped most (9.2%). LM-modified code helped least (5.7%).
Issue text	LM-written issues worked as well as real ones (7.7% and 7.8%).
Difficulty	Harder tasks did not clearly give better models.
Repositories	More repositories helped. Scores rose roughly with the log of the count.

SWE-Gym and SWE-smith side by side

Property	SWE-Gym	SWE-smith
Task source	Real issues	Made bugs
Tasks	2,438	50,137
Repositories	11	128
Environment	One per task	One per repository
Storage	6 TB	295 GB
Human time	About 200 hours	About 20 hours

A caution: what SWE-bench Verified measures

~90%

of tasks are small fixes, under an hour for an expert

12

repositories supply all 500 tasks

5–10%

estimated share of tasks with errors

Both papers report their gains on this benchmark. Read the numbers with these limits in mind.

Six design rules from both papers

1. Grade with tests that fail before the fix and pass after it.
2. Share one environment across many tasks to cut cost.
3. Check every task automatically before you keep it.
4. Add more repositories before you add harder tasks.
5. Cap passing rollouts per task, so easy tasks do not dominate.
6. Keep benchmark repositories out of your training tasks.

How students can try it

1 · Look	`load_dataset("SWE-bench/SWE-smith")`
2 · Run one	Start one task’s Docker container with the SWE-smith library.
3 · Evaluate	Run an agent on 20 tasks and score it with the tests. No GPU needed.
4 · Make tasks	Point SWE-smith at a new repository and make your own bugs.

An Indian version: project ideas

Tasks from Indian open-source Python projects

Use the SWE-smith method on repositories that India’s developers use and build.

Issue text in Hindi or Hinglish

Same bug, same tests, new language. Does the solve rate drop?

Bugs in Indic text-processing code

Transliteration, script handling and tokenizers: the tests are easy to write.

Questions for discussion

1. What kinds of real bugs can bug injection never produce?
2. The issue writer sees the fix. How do you stop it from leaking the answer?
3. How could a model pass the tests without a real fix?
4. Why might more repositories help more than harder tasks?

Read the originals

- SWE-Gym paper (ICML 2025)
- SWE-Gym repository and data and models
- SWE-smith paper (NeurIPS 2025 spotlight)
- SWE-smith repository and documentation
- Epoch AI: What skills does SWE-bench Verified evaluate?